G Minder Milestone 1

Design document

Carlos David Urra Cabello | Student No. 3125350
Mobile Development | Griffith College Dublin | 6 November 2026

1 Main idea and purpose

G-Minder is an Android reminder application designed around tasks associated with places. The sample tasks reflect my routine: returning library books at Griffith College, checking my work rota at Roly's Bistro, buying groceries at Tesco in Stillorgan, and collecting coffee beans at Mister Magpie in Ballsbridge. The intended final application will use location awareness to deliver reminders at relevant places. Milestone 1 demonstrates this journey using predefined locations.

2 Objectives and implemented scope

The milestone objectives are to provide distinct functional layouts, support meaningful user input, demonstrate explicit and implicit Android intents, and establish consistent navigation and styling. The current implementation contains four screens: My Reminders, Create Reminder, Choose Location and Reminder Details.

Users can enter a title and optional notes, select a predefined place, save a reminder in app state, view its details, and move it between Active and Completed. A Maps action opens the selected place in a compatible external application. An ambient-light sensor switches the interface between light and dark themes. At this stage, features are accessed directly through the main cards and their associated screens, including the theme control on the home screen.

GPS monitoring, geofencing, reminder notifications, permanent reminder storage and user profiles remain outside this milestone. The selected places are examples rather than measured positions. Appearance preferences are saved locally; reminder data currently uses UI state rather than a database.

3 Navigation and interaction

Create: My Reminders → Create Reminder → Choose Location → Create Reminder → My Reminders.
Review: My Reminders → Reminder Details → Maps or update status → My Reminders.

The first three layouts share MainActivity and are selected through a screen state value. Reminder Details uses a separate activity launched by an explicit intent. Back returns from the location selector to the draft and from the form to the list. The draft title and notes remain available while selecting a place.

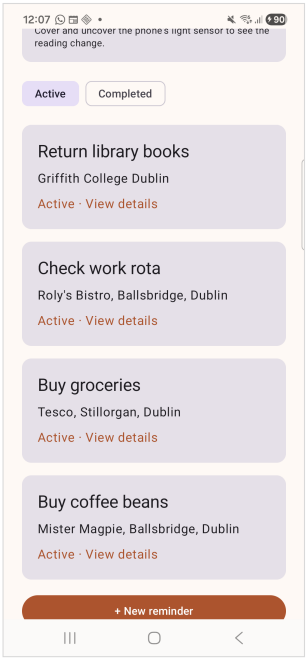
4 Visual design decisions

The interface uses a warm light background, orange accents, rounded cards and Material 3 controls. Large page headings establish hierarchy; full-width actions make the next step easy to find. Primary filled buttons identify actions such as Save, while outlined buttons distinguish supporting actions such as opening Maps. Shared spacing and the G-MINDER label maintain continuity across screens.

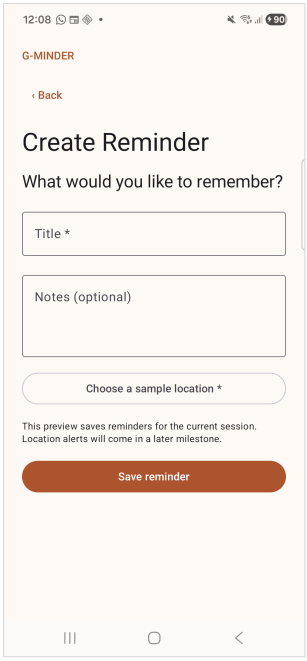
The implemented palette differs from the initial Figma mock-ups. The layout and navigation concepts were retained, while the colours were revised to create a consistent appearance across the light and sensor-controlled dark themes.

Screen design and usability

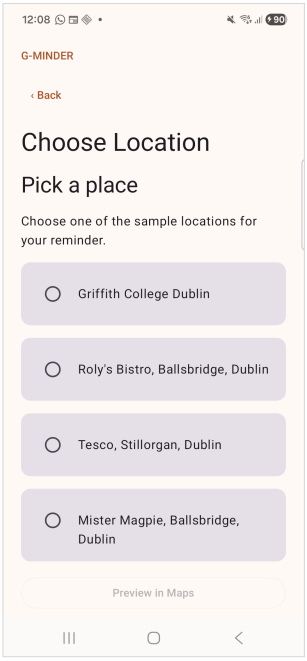
Figure 1. Screenshots captured from the running G-Minder app on the Samsung phone, 6 November 2026. The home screen scrolls to reveal additional reminders and the New reminder action.



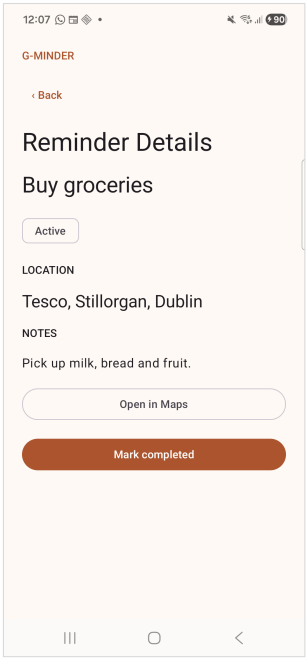
My Reminders



Create Reminder

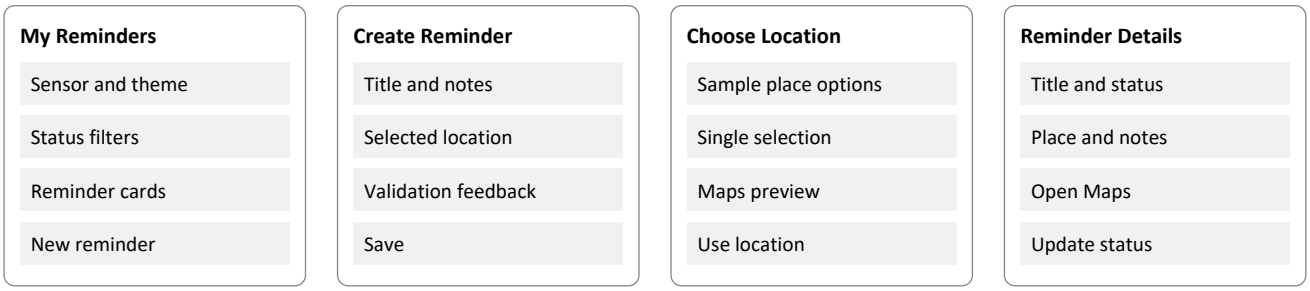


Choose Location



Reminder Details

Figure 2. Schematic wireframes summarising the hierarchy of each screen. These are layout diagrams, separate from the device screenshots above.



5 Layout choices

My Reminders: cards group the task title, place and status. Active and Completed filters reduce clutter. The sensor card exposes the current lighting and theme control. Scrolling keeps lower content and the New reminder button reachable.

Create Reminder: title, notes, location and Save follow the order in which a task is entered. Required fields are marked. Save validates the title and place; missing information produces visible feedback instead of adding an incomplete record.

Choose Location: radio buttons indicate a single choice. Preview and confirmation actions remain disabled until a place is selected. **Reminder Details:** title, status, location and notes precede the Maps and completion actions, supporting review before action.

The shared Page composable applies consistent padding and typography. Safe drawing and keyboard padding reduce obstruction by system controls, while vertical scrolling supports smaller displays. The adaptive dark palette uses lighter accents on a dark background without changing the phone's brightness setting.

Implementation and evaluation

6 Kotlin and Compose structure

The app uses Kotlin and Jetpack Compose. MainActivity.kt defines the Reminder model, main layouts, shared Page composable, theme and Details activity. AmbientLightCard.kt displays sensor state and a switch callback. LightAwareActivity.kt supplies shared sensor handling to both activities.

Reminder records contain an ID, title, note, place and completion flag. Observable state updates the interface after typing, selection and completion. rememberSaveable and a custom saver support restoration of small UI values during activity recreation; they do not replace persistent reminder storage [1].

Code example 1. Layout excerpt from the project's custom Page composable; the body is shortened. The shared container provides keyboard-aware scrolling and consistent spacing.

```
@Composable
fun Page(title: String, back: (() -> Unit)? = null,
         content: @Composable ColumnScope.() -> Unit) {
    Surface(Modifier.fillMaxSize()) {
        Column(Modifier.safeDrawingPadding().imePadding()
            .verticalScroll(rememberScrollState()).padding(24.dp),
            verticalArrangement = Arrangement.spacedBy(16.dp)) {
            /* shared heading and Back button */
            content()
        }
    }
}
```

7 Intents and sensor behaviour

The explicit intent names ReminderDetailsActivity and transfers reminder values through extras. A returned result contains the ID and updated status. The implicit ACTION_VIEW intent uses a geo URI to display the selected place; Android selects a capable application [2]. A missing Maps handler is caught and explained with a message.

The light sensor is checked for availability, registered when an activity resumes and unregistered when it pauses [3]. With adaptation enabled, readings below 30 lux select dark mode and readings above 80 select light mode. Values between those limits retain the previous theme to reduce flicker. The switch choice and lighting classification are stored in SharedPreferences.

Code example 2. Theme selection in LightAwareActivity.kt. The separate boundaries avoid repeated changes around one cutoff.

```
val nextDark = when {
    lux < 30f -> true
    lux > 80f -> false
    else -> darkSurroundings
}
```

8 Evaluation and next steps

The application has built and run on a Samsung Android phone. Manual checks confirmed reminder creation, Maps opening and completion updates. Final checks before submission should cover empty fields, Back navigation, scrolling, saved appearance preferences and dark/light switching on both activities. Missing-sensor and missing-Maps cases have fallback handling; they have not been independently verified on every device.

Later milestones will develop location detection, notifications and permanent reminder storage. A dedicated Settings card and a hamburger menu are proposed to group secondary controls, including theme preferences and future profile or app-lock settings. This would keep the main reminder cards focused on tasks. The Settings card, menu, profiles and app lock are not yet implemented.

References

[1] Android Developers. Save UI state in Compose. developer.android.com/develop/ui/compose/state-saving
[2] Android Developers. Intents and intent filters. developer.android.com/guide/components/intents-filters
[3] Android Developers. Sensors overview. developer.android.com/develop/sensors-and-location/sensors/sensors_overview
Implementation excerpts and wireframes: G-Minder project. Documentation accessed 6 November 2026.